

SUPABASE

vs Python / FastAPI

Rapport technique complet pour choisir
la meilleure architecture pour notre projet

Ce rapport présente une analyse approfondie de Supabase — ses fonctionnalités, ses avantages, ses limites — et le compare objectivement à une architecture Python / FastAPI / PostgreSQL pour aider l'équipe à faire le meilleur choix technique.

PROJET	DevisEvent — Générateur de devis événementiel
STACK COMPARÉE	Supabase vs FastAPI + PostgreSQL
DATE	15 April 2026
AUTEUR	Équipe technique DevisEvent

Sommaire

1.	Qu'est-ce que Supabase ?	2
2.	Les 4 piliers de Supabase	3
3.	Supabase vs Python / FastAPI — Comparaison complète	4
4.	Les langages utilisés avec Supabase	5
5.	Architecture de notre projet DevisEvent	6
6.	Avantages concrets pour notre deadline	7
7.	GitHub, déploiement et hébergement	8
8.	Les limites de Supabase — soyons honnêtes	9
9.	Quand utiliser FastAPI à la place	9
10.	Verdict et recommandation finale	10

1

Qu'est-ce que Supabase ?

Définition : Supabase est une plateforme open-source qui fournit une base de données PostgreSQL hébergée dans le cloud, accompagnée d'un ensemble complet d'outils backend (authentification, stockage de fichiers, sécurité) prêts à l'emploi et accessibles directement depuis le frontend via une bibliothèque JavaScript.

Pour comprendre Supabase, partons de ce que l'équipe connaît déjà : **PostgreSQL**. Supabase c'est exactement PostgreSQL, mais au lieu d'être installé localement sur votre machine, il est hébergé sur des serveurs dans le cloud et accessible depuis n'importe où dans le monde. Mais Supabase va bien plus loin qu'une simple base de données en ligne.

Souvent comparé à **Firestore de Google**, Supabase a la particularité d'être **entièrement open-source** et de reposer sur PostgreSQL — une technologie reconnue, éprouvée et que les développeurs maîtrisent déjà. Là où Firestore utilise une base NoSQL propriétaire, Supabase utilise du SQL standard.

L'analogie qui résume tout :

FastAPI + PostgreSQL c'est comme une cuisine où vous devez tout cuisiner vous-même.



Supabase c'est un restaurant tout équipé où le plat est déjà prêt.

2

Les 4 piliers de Supabase

■ Authentification (Auth)

Gère l'inscription, la connexion et les sessions utilisateurs automatiquement.

- Inscription par email + mot de passe en 2 lignes de code
- Connexion avec gestion automatique des tokens JWT
- Déconnexion et renouvellement des sessions
- Compatible OAuth : connexion via Google, GitHub, Facebook...
- Système de récupération de mot de passe intégré

```
supabase.auth.signInWithPassword({ email, password })
```

■ Base de données (Database)

PostgreSQL hébergé dans le cloud avec une API REST générée automatiquement.

- C'est du PostgreSQL pur — les mêmes tables et requêtes SQL qu'en local
- Interface visuelle pour créer les tables sans écrire de SQL
- API REST générée automatiquement depuis les tables
- Requêtes en temps réel — les données se mettent à jour sans recharger
- Jointures, index, triggers : toutes les fonctionnalités PostgreSQL

```
supabase.from('quotes').select('*', clients(*), quote_items('*'))
```

■ Stockage de fichiers (Storage)

Système de stockage d'images, vidéos et fichiers avec URLs publiques.

- Upload de fichiers PNG, JPG, PDF directement depuis le navigateur
- Organisation en buckets (dossiers) — ex: 'business-assets'
- Génération automatique d'URLs publiques pour accéder aux fichiers
- Contrôle d'accès par bucket : public ou privé
- Idéal pour stocker les logos et tampons des entreprises

```
supabase.storage.from('business-assets').upload('logo.png', file)
```

■ Sécurité (Row Level Security)

Système de règles SQL qui contrôle l'accès aux données au niveau de la base.

- RLS (Row Level Security) : chaque ligne de donnée a ses propres permissions
- Une règle SQL suffit pour isoler complètement les données par utilisateur
- Impossible pour un utilisateur d'accéder aux données d'un autre
- La sécurité est appliquée côté base de données, pas côté code
- Résiste aux failles de sécurité côté frontend

```
CREATE POLICY 'user_policy' ON quotes FOR ALL USING (org_id = auth.uid())
```

3

Supabase vs Python / FastAPI — Comparaison

Voici une comparaison objective et honnête entre les deux approches. Cette comparaison est faite dans le contexte précis de notre projet DevisEvent avec les contraintes de temps et de complexité que nous avons.

Critère	Supabase	FastAPI + PostgreSQL	Avantage
Mise en place	Compte créé → base en ligne en 5 min	Installer Python, configurer DB, créer les models...	✓ Supabase
Authentification	Intégrée — 2 lignes de code	Coder JWT, middleware, sessions manuellement	✓ Supabase
Routes API	Générées automatiquement	Coder chaque route GET/POST/PUT/DELETE	✓ Supabase
Stockage fichiers	Storage intégré en 3 lignes	Coder système d'upload + serveur de fichiers	✓ Supabase
Sécurité données	RLS automatique via SQL	Vérifications manuelles dans chaque route	✓ Supabase
Hébergement backend	Déjà en ligne — rien à faire	Déployer sur Render/Railway/VPS	✓ Supabase
Contrôle du code	Limité aux règles Supabase	Contrôle total sur chaque ligne	✓ FastAPI
Logique complexe	Edge Functions (limité)	Python pur — aucune limite	✓ FastAPI
Performance avancée	Optimisée pour cas standards	Optimisable à l'infini	✓ FastAPI
Coût en prod	Gratuit jusqu'à 500MB	Gratuit sur certains hébergeurs	= Égalité
Courbe d'apprentissage	SQL + 5 méthodes JS	Python + FastAPI + DevOps	✓ Supabase
Temps de développement	Très rapide (jours)	Long (semaines)	✓ Supabase

Score pour notre projet : Supabase remporte **9 critères sur 12**. FastAPI est meilleur sur le contrôle du code, la logique complexe et les performances avancées — des aspects qui ne sont pas critiques pour notre générateur de devis.

4 Les langages utilisés avec Supabase

C'est l'une des questions les plus importantes pour l'équipe. Voici exactement quels langages on utilise et pour quoi.

SQL	JavaScript	Rien !
Pour créer les tables	Pour l'interface React	Pour le backend
Les mêmes commandes CREATE TABLE, SELECT, INSERT que vous connaissez avec PostgreSQL. Rien de nouveau à apprendre.	La bibliothèque supabase-js permet d'appeler la base de données directement depuis React en JavaScript. Une ligne remplace toute une route Python.	Avec FastAPI il faudrait coder des dizaines de fonctions Python. Avec Supabase : zéro. L'API est générée automatiquement.

Exemple concret — Récupérer les devis :

Avec FastAPI (Python)	Avec Supabase (JavaScript)
<pre>from fastapi import Depends from sqlalchemy.orm import Session @app.get('/quotes') async def get_quotes(db: Session = Depends(get_db), user = Depends(get_current_user)): return db.query(Quote) .filter(Quote.org_id == user.org_id) .all() # + fichier models.py # + fichier schemas.py # + fichier database.py # + gestion des tokens # + déploiement serveur</pre>	<pre>// Dans le composant React const { data: quotes } = await supabase .from('quotes') .select('*') .eq('org_id', orgId) // C'est tout. // Auth : automatique // Sécurité : RLS gère tout // Hébergement : Supabase Cloud // Backend : 0 ligne de code</pre>

5

Architecture de notre projet DevisEvent

Voici comment les différentes parties de notre application s'assemblent. L'architecture est volontairement simple : deux services dans le cloud qui communiquent directement, sans couche intermédiaire.

■ UTILISATEUR

L'organisateur événementiel sur son téléphone ou ordinateur. Il crée les devis, uploades son logo et son tampon, puis télécharge le PDF.

↕ *Navigateur web*

■ FRONTEND — React + Vite

Interface utilisateur développée en React. Contient les pages (connexion, profil, nouveau devis, aperçu PDF), le style Tailwind CSS et la logique PDF avec html2canvas + jsPDF. Hébergé sur Vercel (gratuit).

↕ *supabase-js*

■ BACKEND — Supabase Cloud

Base de données PostgreSQL + Auth + Storage + Sécurité RLS. Hébergé sur les serveurs de Supabase. Toujours en ligne, gratuit jusqu'à 500MB.

■ Point clé

Il n'y a PAS de serveur backend à démarrer, déployer ou maintenir. Supabase est déjà en ligne dès la création du projet. Le frontend React communique directement avec Supabase via supabase-js — une bibliothèque JavaScript de 3 lignes d'initialisation.

6

Avantages concrets pour notre deadline

Voici les avantages de Supabase exprimés en **heures de travail économisées** pour notre projet précis. Ces estimations sont réalistes pour une équipe qui découvre les deux approches.

Fonctionnalité	Avec FastAPI	Avec Supabase	Gain
Système d'authentification	1 à 2 jours	30 minutes	~12h
Routes API (CRUD complet)	2 à 3 jours	0 heure	~20h
Stockage des logos/tampons	4 à 6 heures	1 heure	~5h
Déploiement du backend	4 à 8 heures	0 heure	~6h
Sécurité et isolation des données	1 jour	2 heures	~6h
Configuration base de données	2 heures	30 minutes	~1.5h
	Total estimé	Total estimé	
	~8 à 12 jours	~2 jours	~50h

Une question importante de l'équipe : qu'est-ce qu'on met sur GitHub ? Et si on clone le projet sur un autre ordinateur, le backend disparaît-il ?

Ce qui va sur GitHub :

■ À commiter	■ À ne JAMAIS commiter
• src/ — tout le code React • public/ — fichiers statiques • package.json — liste des bibliothèques • tailwind.config.js — configuration • vite.config.js — configuration • .gitignore — fichier de protection • README.md — documentation	• .env.local — contient les clés secrètes Supabase. Si publié sur GitHub, n'importe qui peut accéder à toute la base de données ! • node_modules/ — trop lourd (200MB+), se réinstalle automatiquement avec <code>npm install</code> • dist/ — fichiers de build générés auto

Si on clone le projet sur un nouvel ordinateur :

Le backend Supabase **ne disparaît jamais**. Il vit en permanence dans le cloud sur les serveurs Supabase, indépendamment du code. Cloner le projet ne change rien aux données, tables ou fichiers stockés. C'est comme une télévision et sa télécommande — perdre la télécommande (le code) ne fait pas disparaître la télé (Supabase).

1	<code>git clone https://github.com/username/dev-is-event.git</code>	Récupère le code React depuis GitHub
2	<code>cd devis-event && npm install</code>	Installe toutes les bibliothèques (React, supabase-js...)
3	Créer <code>.env.local</code> avec les clés Supabase	Les clés se trouvent dans Settings > API sur supabase.com
4	<code>npm run dev</code>	L'app se reconnecte à Supabase. Toutes les données sont là.

8

Les limites de Supabase — soyons honnêtes

Un rapport honnête doit aussi parler des inconvénients. Supabase n'est pas parfait pour tous les cas d'usage.

■ Plan gratuit — mise en veille

Sur le plan gratuit, Supabase met le projet en pause après 7 jours d'inactivité. Il faut se connecter au tableau de bord et cliquer sur 'Restore project'. Le plan Pro (25\$/mois) supprime cette limitation.

■ Dépendance à un tiers (Vendor lock-in)

Si Supabase ferme ou change ses tarifs, migrer vers un autre hébergeur PostgreSQL demande du travail. Avec FastAPI, tout le code vous appartient complètement.

■ Logique métier complexe limitée

Pour des calculs complexes, de la manipulation avancée de données ou des intégrations spécifiques, il faut utiliser les 'Edge Functions' de Supabase (du JavaScript côté serveur). Avec FastAPI, Python pur n'a aucune limite.

■ Pas adapté aux apps très spécifiques

Pour des projets avec des besoins très particuliers (machine learning, traitement d'images côté serveur, intégrations complexes), FastAPI + Python reste plus adapté.

■ Connexion internet requise

L'app ne fonctionne pas sans internet car tout est dans le cloud. Une version offline n'est pas possible nativement.

9

Quand utiliser FastAPI à la place ?

FastAPI reste un excellent choix dans certains contextes. Voici les cas où FastAPI serait préférable à Supabase.

Choisir FastAPI si...

Choisir Supabase si...

- Logique métier très complexe (calculs, IA, ML)
- Intégrations spécifiques (APIs tierces complexes)
- Besoin d'un contrôle total sur chaque ligne de code
- Projet à long terme avec grosse équipe backend
- Application nécessitant du traitement temps réel lourd
- Contraintes de conformité (données sur vos propres serveurs)

- Projet avec deadline courte (comme le nôtre)
- CRUD standard : créer, lire, modifier, supprimer des données
- Petite équipe sans développeur backend dédié
- MVP ou prototype à valider rapidement
- Application mobile-first avec auth et stockage de fichiers
- Budget limité (plan gratuit très généreux)

Pour le projet DevisEvent, avec la deadline de lundi et les fonctionnalités requises (authentification, stockage de logos/tampons, gestion de devis, export PDF), **Supabase est objectivement le meilleur choix**. Il économise entre 50 et 80 heures de développement backend sans sacrifier la qualité du produit final.

- ✓ Tu connais déjà PostgreSQL → les tables Supabase se créent avec le même SQL
- ✓ Pas de Python à coder pour le backend → tout en JavaScript dans React
- ✓ Hébergement gratuit immédiat → l'app est en ligne dès le premier jour
- ✓ Le code va sur GitHub, le backend reste dans le cloud Supabase
- ✓ Si on clone le projet → Supabase est toujours là, juste recréer .env.local
- ✓ Plan gratuit largement suffisant pour notre projet et la présentation

→ FastAPI reste excellent pour des projets plus complexes avec plus de temps

**Supabase = FastAPI + PostgreSQL + Hébergement + Auth + Storage
en un seul outil, gratuit, déjà en ligne, prêt en 30 minutes.**